

Go Programming Master Course

Epic Learn Education · 16 weeks (4 months) · Beginner → junior backend

Instructor: Umesh Indrajith · BSc Eng (Hons) Electronic & Telecommunication,
University of Moratuwa · Senior Software Engineer / Lecturer

This document is the teaching curriculum. It extends the original 16-week outline with the language and software-engineering topics that were missing, while keeping the original design choice: **students start the Student Management System (SMS) demo in weeks 4–5 and get 10–12 weeks of hands-on project time.**

Teaching model

This is a **software engineering course that uses Go**, not a language-tour that ends with a project.

Spiral, not waterfall

Weeks 1–3 build enough Go to be dangerous. Weeks 4–16 grow **one product**. When a language topic appears *after* REST (functions, structs, pointers, errors), that is intentional:

1. **Just enough** in the week they need it to ship a vertical slice (copy a handler, call a function, return JSON).
2. **Deepen** the same topic the following week *inside the SMS code they already own*.

Students should never sit through four weeks of syntax with nothing to demo. They should also never be asked to invent layered architecture without having seen a function.

Demo project: Student Management System (this repo)

The class product is **SMS** — a campus admin API plus a simple React client:

Area	What students build
Product	Staff register/login, then create / list / update / delete students
Backend (assessed)	Go 1.25, Gin, layered <code>cmd/</code> + <code>internal/</code> , PostgreSQL, JWT
Frontend (provided)	React + Vite client in <code>frontend/</code> . Go students run it and wire CORS; they are not graded as React developers

Area	What students build
Target tree	<code>backend/cmd/server</code> , <code>internal/handlers</code> , <code>internal/repository</code> , <code>internal/models</code> , <code>internal/middleware</code> , <code>internal/routes</code> , <code>internal/database</code> , <code>internal/config</code>

Build order in class is not the same as git history. This WIP repo added auth before a finished student UI. In class we do **health** → **in-memory student CRUD** → **layers/errors** → **concurrency extras** → **tests** → **PostgreSQL** → **JWT/auth** → **frontend wiring** → **polish**. The finished shape is still this repository.

Weekly rhythm (3–4 contact hours + homework)

Block	Time	Purpose
Concept	45–60 min	Why this exists in real systems
Live code	45–60 min	Instructor types; students type along
Lab	60–90 min	SMS milestone or language kata
Homework	3–5 hours	PR on their SMS fork

Every homework is a **GitHub pull request**, not a zip file.

Course objectives

By week 16 a passing student can:

1. Write idiomatic Go: types, zero values, control flow, slices, maps, functions, structs, interfaces, pointers, `defer` , packages, modules.
2. Explain value vs pointer, stack vs heap at a practical level, and why Go has a garbage collector.
3. Design a small HTTP JSON API (routing, status codes, validation, layered handlers/repository).
4. Handle errors with `error` values, wrapping (`%w`), `errors.Is` / `errors.As` , and structured logs (`log/slog`).
5. Use goroutines, channels, `waitGroup` , mutexes, and `context` without data races (`go test -race`).

6. Persist data with PostgreSQL, parameterized SQL, transactions, and simple migrations.
7. Test with `testing`, table-driven tests, and `httptest`; format and vet with `gofmt` / `go vet`.
8. Use Git/GitHub, environment config (no secrets in git), and a single CI workflow.
9. Walk through SMS: request → middleware → handler → repository → database → JSON response.

Honest scope: this is a junior backend foundation with a portfolio API. It is not mastery of distributed systems or cloud platforms. Kubernetes, advanced generics, and ORM internals are out of scope.

Tools and versions (pin these)

Tool	Version / choice
Language	Go 1.25.x (install from go.dev; do not mix versions in class)
OS	Ubuntu Linux lab. Windows students: WSL2 + Ubuntu . macOS is fine
Editor	VS Code + official Go extension (Delve debugger). GoLand optional
HTTP	Teach <code>net/http</code> for one session, then Gin (this repo)
Database	PostgreSQL 16+
API checks	Postman and <code>curl</code>
VCS	Git + GitHub (one repo per student, instructor as collaborator)
Go tools	<code>gofmt</code> , <code>go vet</code> , <code>go test</code> , <code>go test -race</code> , <code>go build</code> , <code>go mod tidy</code>
Logging	<code>log/slog</code> (stdlib)
Frontend	Node 20+ only when running <code>frontend/</code> (week 15)

Framework rule: **stdlib first, Gin for SMS**. Students must be able to explain what Gin is wrapping.

Capstone spec (SMS)

Users

- A staff user registers with first name, last name, username, email, password.
- Login returns a JWT. Protected routes send `Authorization: Bearer <token>`.

Students

- Authenticated staff can create, list, get by id, update, and delete students.
- Student fields: first name, last name, email, phone, course, date of birth.

Quality bar (week 16)

- Layered folders matching this repo
- Errors and logs on every failure path
- At least a few unit tests and one HTTP test
- PostgreSQL with migrations, not “tables I clicked in pgAdmin and forgot”
- README: how to run, env vars, example `curl s`
- No passwords or JWT secrets committed

Stretch (not required to pass)

- Student CRUD UI on `frontend/src/pages/Students.jsx` (currently a stub)
- Dashboard stats (`internal/handlers/dashboard_handler.go` is empty)
- CSV/bulk import using a worker pool (weeks 11–12)

16-week syllabus

Legend: **Language** = Go. **SE** = software engineering. **SMS** = what lands in the project that week.

Week 01 — Software engineering, Go, toolchain, Git

Day 1 materials (full kit): [docs/week-01-day-01/](#) — slides, presenter notes, student handout, lab sheet, install, troubleshooting, homework. **Printable PDFs:** [docs/week-01-day-01/pdf/](#)

Day 1 slides: [docs/slides/week-01-day-01.html](#) · [presenter notes](#)

Language / SE

- What software engineering is: requirements → design → implement → test → review
- Why Go: simplicity, toolchain, concurrency, static binaries, stdlib
- Install Go 1.25, `GOROOT / GOPATH` vs modules, `go env`
- VS Code + Go extension, `gofmt` on save
- `package main`, `func main`, `go run`, `go build`
- Exported vs unexported names (`Main` vs `main` is the wrong example — use `fmt.Println` vs local `println`)
- **Git from day one:** `init`, `clone`, `status`, `add`, `commit`, `.gitignore`, GitHub remote

SMS: none yet. Personal `hello` repo.

Lab: install toolchain; print `Hello, Epic Learn`; commit to GitHub.

Homework: short write-up “why we will use Go for SMS” (half page) + screenshot of `go version`.

Do not skip: `gofmt`. Unformatted code is returned in review from this week on.

Week 02 — Variables, types, zero values, operators, strings

Language

- `var`, `:=`, `const`, `iota` (light)
- Basic types, conversions (no implicit numeric conversion)
- **Zero values** (`0`, `""`, `nil`, `false`) — this is how Go avoids uninitialized junk
- Operators: arithmetic, assignment, comparison, logical, bitwise
- `fmt`, `strings`, runes vs bytes (one short UTF-8 demo: length of `"සිංහල"` or `"café"`)
- `time` basics (dates return in SMS later)

SE: naming (`camelCase`, `MixedCaps` for export), comments that explain *why*.

SMS: none.

Lab: type-conversion and zero-value worksheet; small CLI that reads flags or stdin.

Homework: 10 focused exercises (types + strings). First use of `go test` on a pure function `func FullName(first, last string) string` — they will not master testing yet;

they see the command.

Week 03 — Control flow, arrays, slices, maps (and a first real function)

Language

- `if`, `else`, `switch` (tagless switch)
- `for` as the only loop; `break` / `continue`; `range`
- Arrays vs slices; `len` / `cap`; `make`; `append` and the **backing-array gotcha**
- Maps: comma-ok, ranging, “key missing”
- **Functions enough to survive week 5:** `func`, parameters, multiple return values, `return x, err` as a *pattern they copy*, not a full theory week
- Variadic `...` (light), closures (light)

Week 3 is dense. Prefer **two labs** (control+slice, then maps) over rushing maps in 20 minutes.

SE: table of test cases on paper for a slice helper.

SMS: none. These katas become SMS utilities later (filter students by course).

Lab: implement `Contains`, `Unique`, and a map-based frequency counter with `go test`.

Homework: slice/map exercises; optional reading: [Effective Go](#).

Week 04 — Modules, packages, project shape, SMS kickoff

Hands-on clock starts here (project week 1 of ~12).

Language / SE

- Packages and imports; `internal/` as a boundary
- Go modules: `go mod init`, `go get`, `go mod tidy`
- Project layout used in this repo:

```
backend/ cmd/server/main.go # process entry internal/config/ # env
internal/handlers/ # HTTP internal/repository/ # data internal/models/ # structs
internal/routes/ internal/middleware/ internal/database/ internal/utils/
```

- Clean code: small files, one package job, no “utils dumping ground” (we still have `utils` for JWT later — call that out as a tradeoff)

- README as the contract of a repo
- Branch + PR workflow; `.env` is gitignored; `.env.example` is committed

SMS milestone: empty module + folders + `GET` not required yet. README: product in three sentences. Fork or recreate this repo's backend skeleton.

Lab: `go mod init student-management-system`; `cmd/server` prints `"sms starting"`.

Homework: PR with folder skeleton and README. Instructor merges only if `gofmt` is clean and `.env` is not committed.

Week 05 — HTTP, JSON, Gin, first SMS API

Why REST this week, before a full functions/structs course: so every remaining week has a running product. Handlers are taught as “functions the router calls.” Deep function theory is week 6; deep structs are week 7.

Language / SE

- HTTP: methods, paths, status codes, headers, JSON body
- `net/http` Hello World (stdlib, 30–40 min) so Gin is not magic
- Gin router, `gin.Context`, `c.JSON`
- `encoding/json` and struct tags — **minimal** struct (id + message) only
- Validation as “reject bad input with 400”
- Postman + `curl`
- CORS mentioned, not implemented yet (frontend is later)

SMS milestone:

- `GET /health` → `{ "status": "ok", "message": "Backend is running" }`
(see `backend/internal/routes/routes.go`)
- `GET /api/hello` or similar JSON
- Run on `:8080`

Lab: health endpoint + Postman collection started.

Homework: screenshot of Postman 200 + PR. Stretch: `GET /api/students` returns a **hard-coded** JSON array (no DB).

Week 06 — Functions and methods (inside SMS)

Language

- Parts of a function: name, params, results, named results (use sparingly)
- Pass by value; why a method needs a receiver
- Multiple returns; ignoring with `_` (and why ignoring `err` is a bug)
- `defer` (function exit: close, unlock, recover-later)
- Methods vs functions; pointer vs value receivers (preview of week 8)
- Generics: **one** example `( func First[T any](s []T) (T, bool) )`, then stop

SMS milestone: extract helpers from `main` into handler functions; `NewStudentHandler` - style constructor as a function that returns a struct they do not fully understand yet.

Files to teach toward: `backend/internal/handlers/*.go` — handlers are methods with `(h *StudentHandler)`.

Lab: refactor week 5 into `handlers` package; add `FullName(first, last string) string` used by a student response later.

Homework: PR refactor. No new product feature required if the refactor is real.

Week 07 — Structs, JSON tags, interfaces, in-memory students

Language

- Structs, composite literals, exported fields
- Methods on structs; embedding (composition, not inheritance)
- JSON tags ``json:"first_name"`` matching this repo's models
- Interfaces: small ones (`error` is an interface); **accept interfaces, return structs**
- `io.Reader` / `io.Writer` as the canonical interface story (request body is a reader)

SMS milestone: `models.Student` and in-memory store (`map[int]Student` or slice).

- `POST /api/students`, `GET /api/students`, `GET /api/students/:id`
- No JWT yet. No PostgreSQL yet.

Files: `backend/internal/models/student.go` (target fields). Ignore DB tags until week 14.

Lab: create + list students via Postman.

Homework: update + delete. Invalid JSON → 400.

Week 08 — Pointers, memory, repository layer

Language

- Pointers, `&` / `*`, nil pointers
- Value vs reference (maps and slices already share backing data)
- Stack vs heap, escape analysis at a *picture* level, GC in one slide
- Why handler methods use pointer receivers (`h *StudentHandler`)
- `make` vs `new` (only if it comes up)

SE: repository pattern — HTTP must not contain SQL (or map access forever).

SMS milestone: `StudentRepository` in memory; handlers call the repo. Matches `backend/internal/repository/student_repository.go` shape, without `pgx`.

Lab: nil-pointer crash demo, then fix; move map into repository.

Homework: PR with handler → repository. Memory diagrams in the PR description (photo of notebook is fine).

Week 09 — Errors, wrapping, validation, slog, JWT preview

Language / SE

- `error` as a value; sentinel vs custom types
- `fmt.Errorf("...: %w", err)`, `errors.Is`, `errors.As`
- Do not panic in HTTP handlers
- Validation (Gin `binding` + this repo's `RegisterRequest` / `CreateStudentRequest`)
- `log/slog` JSON logs: level, msg, attrs (`student_id`, `err`)
- Config from environment (`internal/config`, `godotenv`) — secrets not in git
- **Auth as product, still light:** password hashing idea (`bcrypt`); JWT signed string. Full middleware can land this week or week 15 if the group is slower — prefer **register/login without protecting student routes yet** if time is tight.

SMS milestone: every handler returns a consistent `{ "error": "..."}` JSON; logs on 5xx. Optional: `POST /api/register` storing users in memory.

Files: `handlers/auth_handler.go` , `models/user.go` , `config/config.go` .

Lab: custom `ErrNotFound` ; map to 404 vs 500.

Homework: replace `fmt.Println` debugging with `slog` . Add `.env.example` .

Week 10 — Goroutines and the scheduler

Language

- Concurrency vs parallelism
- Goroutines; `go f()`
- Go scheduler (GMP) at poster level
- Leaking goroutines; why HTTP handlers must not `go` work without a plan

SMS milestone (practical, not theatre):

- Add a **timeout-friendly** slow path or dashboard aggregation later
- Or: log student creates asynchronously *only after* they see the race you will fix in week 12

Safer lab: fan-out three independent validations with goroutines and collect errors — then discuss why this may be overkill for SMS.

Lab: 10k goroutines printing vs a data race on a counter (`go test -race`).

Homework: short quiz + optional race in their in-memory repo if they used a global map unsafely.

Week 11 — Channels, pipelines, worker pool

Language

- Unbuffered vs buffered vs directional channels
- `select` , `close` , `range` over channel
- Pipeline pattern
- **Worker pool** (the one pattern they should remember)

SMS stretch feature: `POST /api/students/import` accepts a JSON array; a pool of N workers validates/inserts; response is `{ created, failed }` .

If the group is struggling, do the worker pool as a **kata** (thumbnail resizer / hash list) and only sketch the import API.

Lab: worker pool kata with `waitGroup` preview.

Homework: import endpoint or documented kata in `backend/internal/workers/` (delete before production if unused).

Week 12 — Mutex, WaitGroup, context, races

Language / SE

- `sync.Mutex` , `RWMutex`
- `sync.WaitGroup`
- `context` : cancel, deadline, timeout; **request-scoped context** on every handler
- `c.Request.Context()` in Gin
- Data races; `go test -race`
- Graceful shutdown sketch (`signal.Notify` , cancel root context)

SMS milestone: lock the in-memory map; pass `context.Context` into repository method signatures (even if ignored until week 14). Timeouts on a demo endpoint.

Lab: race detector red → mutex → green.

Homework: PR with `-race` in the README test command.

Week 13 — Testing and code quality

Language / SE (this is the *deep* testing week; they have seen `go test` since week 2)

- `testing` package, `t.Run` , table-driven tests
- `httptest` for Gin (health + one student handler)
- Fakes: repository interface with an in-memory fake (this is why interfaces exist)
- `go vet` , coverage (`go test -cover`)
- Delve: breakpoint in a handler
- **CI:** GitHub Actions: `gofmt -l` , `go vet` , `go test ./...`

SMS milestone: tests for `FullName` / validation / `GET /health` ; CI badge or green workflow.

Files: new `*_test.go` next to packages. There are **no tests in this WIP yet** — that is a known gap students fill.

Lab: table-driven parse of `YYYY-MM-DD` (see `CreateStudent` date parsing).

Homework: CI workflow + at least 5 tests. Code quality notes in the PR.

Week 14 — PostgreSQL, SQL, CRUD, transactions, migrations

SE / data

- What a relational DB is; SQL: `SELECT` `INSERT` `UPDATE` `DELETE`
- PostgreSQL install on Ubuntu; `psql`
- Drivers: this repo uses `pgx` (`internal/database/database.go`)
- Parameterized queries only (`$1` , `$2`) — **SQL injection demo** with string concat, then the fix
- Transactions (create student + audit later; or register user)
- Migrations as SQL files (not “click in a GUI”)

SMS milestone: replace in-memory student store with PostgreSQL. Schema:

See `backend/migrations/001_init.sql`.

Lab: `createdb sms` ; run migration; `GetAll` from Postman hits Postgres.

Homework: user table + persist register **or** keep users in memory one more week if JWT is week 15. Prefer users in DB this week if register already exists.

Week 15 — Auth, frontend wiring, project completion

SE / product

- JWT (HS256) as this repo does in `internal/utils/jwt.go`
- Bearer middleware (`internal/middleware/auth_middleware.go`)
- Protect student routes; leave `/api/register` and `/api/login` public
- bcrypt cost; never log passwords
- CORS for `http://localhost:5173` (`cmd/server/main.go`)
- Run the provided React app: login, register, dashboard
- Application flow lecture: browser → Vite → Gin → JWT → repository → Postgres

- Debugging production-like issues: 401 vs 404 vs 500, CORS, wrong `VITE_API_URL`

SMS milestone: feature-complete backend matching this WIP; frontend login works against their API. Student UI may still be a stub (`frontend/src/pages/Students.jsx`) unless they take the stretch.

Lab: end-to-end register → login → `GET /api/students` with Bearer token in Postman **and** in the React app.

Homework: README runbook (backend, postgres, frontend). Fix at least one real bug found in the lab.

Week 16 — Review, Go design, refactor, demo

SE

- Code review (instructor rubric below)
- Error handling and logging pass
- Refactor: naming, dead code, consistent JSON errors
- **Go design review** (use SOLID only as a comparison, not as the scoring bible):
- Small interfaces
- Composition via embedding
- Accept interfaces, return structs
- Package clarity
- A little copying beats a little dependency
- Optional: Docker Compose `api + postgres` (stretch)
- Final demonstration: 8–10 minutes per student/team

SMS milestone: demo + merged `main` + retrospective (what they would do in a v2: pagination, OpenAPI, refresh tokens).

Homework: none after demo. Portfolio README.

Topic index (original outline → where it lives now)

Original week	Kept?	Notes
1 Intro / setup	Yes	Plus Git and <code>gofmt</code>

Original week	Kept?	Notes
2 Variables / types	Yes	Plus zero values, strings/runes
3 Control / slices / maps	Yes	Plus first functions and <code>go test</code>
4 Modules / structure	Yes	SMS starts
5 REST + project	Yes	Stdlib HTTP then Gin; just-in-time handlers
6 Functions / methods	Yes	Deepen inside SMS
7 Structs / interfaces	Yes	Plus JSON tags, <code>io.Reader</code>
8 Pointers / GC	Yes	Plus repository layer
9 Errors / logging	Yes	Plus wrapping, <code>slog</code> , env config
10 Goroutines	Yes	Race detector intro
11 Channels	Yes	Worker pool; optional bulk import
12 Sync / context	Yes	Mutex on memory store; request context
13 Testing	Yes	Plus <code>httptest</code> , CI, Delve
14 Database CRUD	Yes	Plus migrations, injection, <code>pgx</code>
15 Project completion	Yes	JWT, CORS, React client
16 Refinement	Yes	Go idioms; SOLID as optional lens

Added throughout: Git/PRs, zero values, `defer`, JSON, `slog`, `context` on HTTP, table-driven tests, CI, migrations, bcrypt/JWT, CORS, Delve, WSL note.

Assessment (suggested)

Component	Weight	Evidence
Weekly PRs (weeks 4–15)	40%	Merged or submitted on time, <code>gofmt</code> , no secrets
Mid checkpoint (end of week 8)	15%	In-memory student CRUD + health, live demo 5 min
Tests + CI (week 13)	15%	Green workflow

Component	Weight	Evidence
Final SMS (weeks 14–16)	25%	Postgres + auth + README + oral walkthrough
Professionalism	5%	Review comments, pairing, honesty about AI-generated code

AI policy: students may use Copilot/ChatGPT. They must be able to explain every line they submit. If they cannot, it does not count.

Code review rubric (weeks 8–16)

1. Builds and `gofmt` is clean
2. Errors returned and logged; no panics on bad input
3. No secrets in git
4. HTTP status codes match the outcome
5. SQL uses parameters
6. Names match Go conventions
7. Tests exist for the week's logic (from week 13)
8. PR description says what and why

Instructor notes

- **Frontend is a client, not a second course.** If Ubuntu lab time is short, stay on Postman until week 15.
- **This repo is WIP.** Known gaps to assign, not hide: empty `dashboard_handler.go`, stub Students page, no tests, no migration files (until `backend/migrations/` is added), `log` instead of `slog`, JWT secret in local `.env`.
- **Pace valve:** if REST in week 5 is shaky, slip JWT to week 15 (already the plan) and drop bulk import. Do not slip PostgreSQL past week 14 or the final demo has nothing durable.
- **Teams:** 2 people max. Both must type. Split handler vs repository, not “one codes, one watches.”
- **Readings:** Effective Go; Go Proverbs; official [Tour of Go](#) for weeks 1–3 only.

Run the SMS demo (instructor / week 15)

Backend:

```
cd backend
# create .env from .env.example (never commit real secrets)
go run ./cmd/server
```

PostgreSQL database `sms` must exist; apply `backend/migrations/001_init.sql`.

Frontend:

```
cd frontend
npm install
npm run dev
```

API default: `http://localhost:8080` (`frontend/src/api/axios.js`). UI:
`http://localhost:5173`.

Epic Learn Education · epiclearneducation.com · Hotline / WhatsApp: 074 1200 659